```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

char expstack[MAX][MAX];

// Push operator
void push(char ch)
{
   top++;
   stack[top] = ch;
}

// Pop operator
char pop()
{
   char ch = stack[top];
   top--;
   return ch;
}

// Push expression
void pushExp(char str[])
{
   top++;
   strcpy(expstack[top], str);
}

// Pop expression
void popExp(char str[])
{
   strcpy(str, expstack[top]);
   top--;
}

// Find priority
int priority(char ch)
{
   if (ch == '^')
      return 3;
```

```
  if (ch == '*' || ch == '/')
    return 2;
  if (ch == '+' || ch == '-')
    return 1;
  return 0;
}

// Check operator
int isOperator(char ch)
{
  if (ch == '+' || ch == '-' || ch == '*' ||
    ch == '/' || ch == '^')
    return 1;

  return 0;
}

// 1. Infix to Postfix
void infixToPostfix(char inexp[], char postexp[])
{
  int i = 0, k = 0;
  char ch;
  top = -1;

  while (inexp[i] != '\0')
  {
    ch = inexp[i];

    if (isalnum(ch))
    {
      postexp[k++] = ch;
    }
    else if (ch == '(')
    {
      push(ch);
    }
    else if (ch == ')')
    {
      while (top != -1 && stack[top] != '(')
        postexp[k++] = pop();

      if (top != -1)
        pop();
    }
    else
```

```
    {
      while (top != -1 && stack[top] != '(' &&
           priority(stack[top]) >= priority(ch))
        postexp[k++] = pop();

      push(ch);
    }

    i++;
  }

  while (top != -1)
    postexp[k++] = pop();
  postexp[k] = '\0';
}
```

// 2. Infix to Prefix

```
void infixToPrefix(char inexp[], char prefix[])
{
  char rev[MAX], postexp[MAX];
  int i, n = strlen(inexp);

  for (i = 0; i < n; i++)
  {
    rev[i] = inexp[n - 1 - i];

    if (rev[i] == '(')
      rev[i] = ')';
    else if (rev[i] == ')')
      rev[i] = '(';
  }

  rev[n] = '\0';

  infixToPostfix(rev, postexp);

  n = strlen(postexp);

  for (i = 0; i < n; i++)
    prefix[i] = postexp[n - 1 - i];
  prefix[n] = '\0';
}
```

// 3. Prefix to Infix

```
void prefixToInfix(char prefix[], char infix[])
{
   char op1[MAX], op2[MAX], temp[MAX];
   int i;

   top = -1;

   for (i = strlen(prefix) - 1; i >= 0; i--)
   {
      if (isalnum(prefix[i]))
      {
         temp[0] = prefix[i];
         temp[1] = '\0';
         pushExp(temp);
      }
      else if (isOperator(prefix[i]))
      {
         popExp(op1);
         popExp(op2);

         sprintf(temp, "(%s%c%s)", op1, prefix[i], op2);
         pushExp(temp);
      }
   }
   popExp(infix);
}
```

// 4. Prefix to Postfix

```
void prefixToPostfix(char prefix[], char postfix[])
{
   char op1[MAX], op2[MAX], temp[MAX];
   int i;

   top = -1;

   for (i = strlen(prefix) - 1; i >= 0; i--)
   {
      if (isalnum(prefix[i]))
      {
         temp[0] = prefix[i];
         temp[1] = '\0';
         pushExp(temp);
      }
      else if (isOperator(prefix[i]))
      {
```

```
        popExp(op1);
        popExp(op2);

        sprintf(temp, "%s%s%c", op1, op2, prefix[i]);
        pushExp(temp);
      }
    }
    popExp(postfix);
}



// 5. Postfix to Infix
void postfixToInfix(char postfix[], char infix[])
{
    char op1[MAX], op2[MAX], temp[MAX];
    int i;

    top = -1;

    for (i = 0; postfix[i] != '\0'; i++)
    {
      if (isalnum(postfix[i]))
      {
        temp[0] = postfix[i];
        temp[1] = '\0';
        pushExp(temp);
      }
      else if (isOperator(postfix[i]))
      {
        popExp(op2);
        popExp(op1);

        sprintf(temp, "(%s%c%s)", op1, postfix[i], op2);
        pushExp(temp);
      }
    }
    popExp(infix);
}

// 6. Postfix to Prefix
void postfixToPrefix(char postfix[], char prefix[])
{
    char op1[MAX], op2[MAX], temp[MAX];
    int i;
```

```
  top = -1;

  for (i = 0; postfix[i] != '\0'; i++)
  {
    if (isalnum(postfix[i]))
    {
      temp[0] = postfix[i];
      temp[1] = '\0';
      pushExp(temp);
    }
    else if (isOperator(postfix[i]))
    {
      popExp(op2);
      popExp(op1);
      sprintf(temp, "%c%s%s", postfix[i], op1, op2);
      pushExp(temp);
    }
  }
  popExp(prefix);
}

// Main function
int main(){
  char inexp[MAX], result[MAX];
  int choice;

  do
  {
    printf("\n--- EXPRESSION CONVERSION ---\n");
    printf("1. Infix to Postfix\n");
    printf("2. Infix to Prefix\n");
    printf("3. Prefix to Infix\n");
    printf("4. Prefix to Postfix\n");
    printf("5. Postfix to Infix\n");
    printf("6. Postfix to Prefix\n");
    printf("0. Exit\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    if (choice >= 1 && choice <= 6)
    {
      printf("Enter expression: ");
      scanf("%99s", inexp);
```

```
    }

    switch (choice)
    {
      case 1:
        infixToPostfix(inexp, result);
        printf("Postfix: %s\n", result);
        break;

      case 2:
        infixToPrefix(inexp, result);
        printf("Prefix: %s\n", result);
        break;

      case 3:
        prefixToInfix(inexp, result);
        printf("Infix: %s\n", result);
        break;

      case 4:
        prefixToPostfix(inexp, result);
        printf("Postfix: %s\n", result);
        break;

      case 5:
        postfixToInfix(inexp, result);
        printf("Infix: %s\n", result);
        break;

      case 6:
        postfixToPrefix(inexp, result);
        printf("Prefix: %s\n", result);
        break;

      case 0:
        printf("Program ended.\n");
        break;

      default:
        printf("Invalid choice!\n");
    }

  } while (choice != 0);
  return 0;
}
```